UNIVERSIDAD AUTÓNOMA DE CHIRIQUÍ

Facultad de Economía — Escuela de Ciencias Computacionales

INFORME DE LABORATORIO N.° 1

Preparación del Entorno de Desarrollo

Asignatura	Programación V (PROG 309)
Eje temático	N.° 1 — Introducción a los Patrones Arquitecturales
Laboratorio	N.° 1 — Preparación del Entorno de Desarrollo
Docente	Alexis Flores Franco
Estudiante	Cristian Valdés
Semestre	2026-II
Universidad	Universidad Autónoma de Chiriquí — Facultad de Economía, Escuela de Ciencias Computacionales

1. Introducción

Antes de empezar a programar en este semestre, se decidio dejar el entorno listo para no tener que pensarlo de nuevo hasta diciembre. En la práctica fueron tres cosas encadenadas: que el editor reconociera el JDK correcto, armar un proyecto Maven con la estructura de paquetes que vamos a ver a fondo más adelante (model, view, controller), y dejar un repositorio en Git conectado a GitHub desde el primer commit.

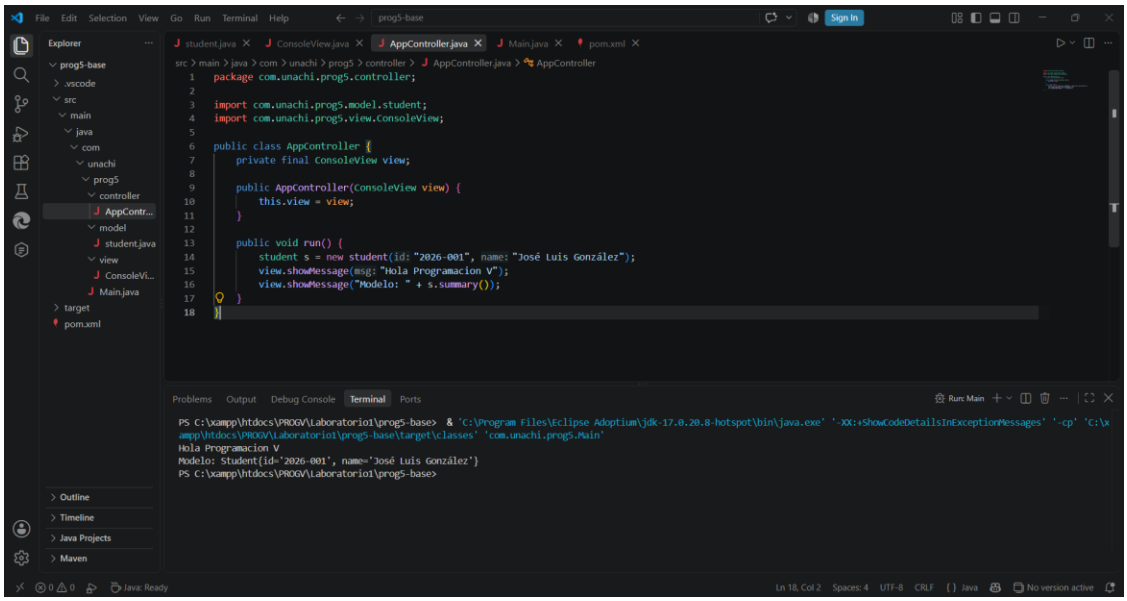
Lo hice en Visual Studio Code en vez de Eclipse. Ya que es lo que uso todos los días para mi trabajo como desarrollador, y prefería que el esfuerzo del semestre fuera entender los patrones, no aprenderme un IDE nuevo. Cada paso queda documentado igual, con el mismo JDK, el mismo Maven, los mismos paquetes y la misma salida esperada en consola.

2. Desarrollo del Laboratorio

2.1 Actividad 1 — JDK 17 y proyecto Maven con estructura MVC

El primer problema al instalar el jdk fue que mi máquina ya tenía instalado el JDK 25, y pensé que con eso bastaba. No fue así. Maven y el laboratorio piden JavaSE-17 sí o sí, así que instalé Eclipse Temurin 17 aparte (los dos JDK conviven) y dejé JAVA_HOME apuntando a la carpeta del 17, no al ejecutable ni a la versión más nueva que ya tenía.

Con eso resuelto, generé el proyecto con el arquetipo maven-archetype-quickstart (groupId com.unachi.prog5, artifactId prog5-base), ajusté el pom.xml para que compilara contra Java 17, y armé los paquetes model / view / controller. Ahí metí las cuatro clases de la guía —Student, ConsoleView, AppController y Main— y las corrí.

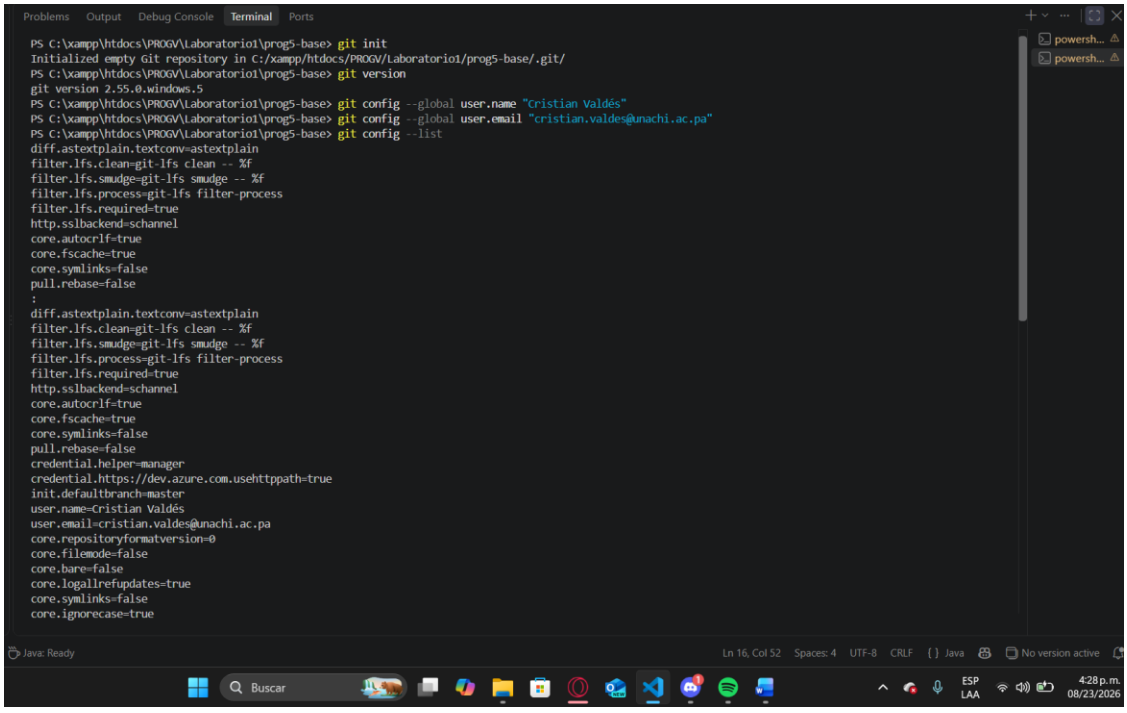


Proyecto Maven corriendo sobre JDK 17, con la salida esperada en consola: "Hola Programacion V" y el resumen del objeto Student.

En la ruta del ejecutable que muestra la terminal (jdk-17.0.20.8-hotspot\bin\java.exe) se ve que el proyecto sí corrió con la versión 17 y no con la 25 que tenía por defecto, que era justo lo que había que comprobar en este paso.

2.2 Actividad 2 — Instalación y configuración de Git

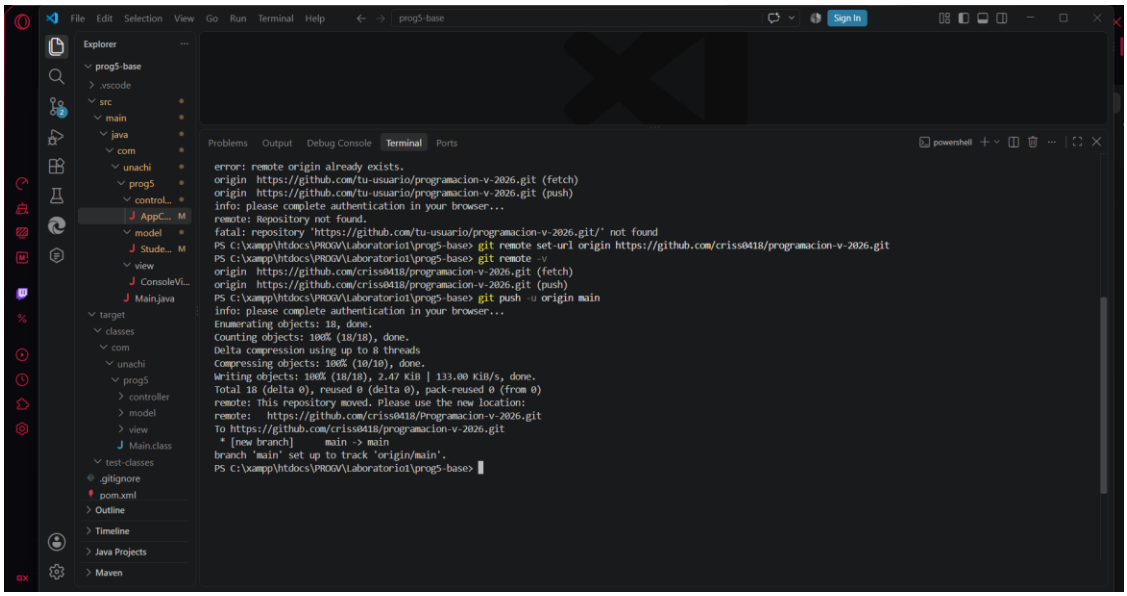
Instalé Git dejando todas las opciones por defecto del instalador y lo configuré con mi nombre y mi correo institucional, que es lo que va a quedar pegado a cada commit de aquí en adelante.



Inicialización del repositorio local (git init), verificación de versión y configuración de user.name y user.email confirmada con git config --list.

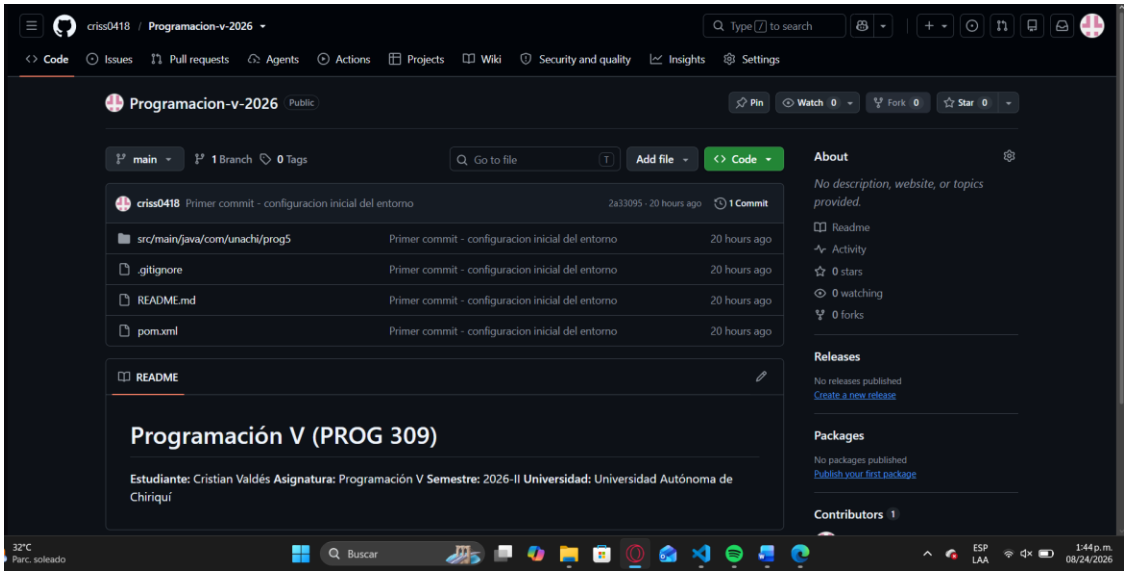
2.3 Actividad 3 — Repositorio de trabajo y conexión con GitHub

Aquí me tropecé de nuevo, pero esta vez fue más despiste que otra cosa: al conectar el remoto dejé sin cambiar el placeholder de la URL de ejemplo, así que el primer push falló con "repository not found". Lo arreglé con git remote set-url apuntando a mi usuario real, y repetí todo: remote add, branch -M main, y push -u origin main.



Corrección de la URL remota y push exitoso: la rama main queda enlazada con origin/main.

Antes del primer commit dejé un .gitignore para que la carpeta target/ y los archivos compilados no se subieran — el repositorio debe quedar solo con código fuente, no con basura generada. Así se ve ya en GitHub:



Repositorio programacion-v-2026 en GitHub, con el primer commit y el README.md visibles.

2.4 Actividad 4 — Reconocimiento visual del patrón de capas

Para esta parte elegí Discord, porque la uso todos los días y es fácil distinguir qué se ve, qué se decide y qué se guarda:

- Presentación: la lista de servidores, los canales, la ventana de chat y el panel de miembros — todo lo que toco yo directamente.
- Negocio: revisar si tengo permiso para escribir en un canal, validar el mensaje, frenar el spam y procesar menciones antes de que le lleguen a alguien.
- Datos: el historial de mensajes, los perfiles, los roles y la lista de amigos — lo que queda guardado y se consulta cada vez.

Lo que quise que se notara en el diagrama es que la presentación nunca toca los datos directamente: todo pasa primero por la capa de negocio, que es la que decide si algo es válido antes de leer o escribir nada.



Figura 5. Diagrama de tres capas aplicado a Discord, elaborado en draw.io.

3. Preguntas de Análisis

3.1 ¿Por qué es importante llevar un control de versiones desde el primer proyecto y no solo en proyectos grandes?

Porque el hábito se agarra desde el principio, no cuando ya urge. Si esperas a que el proyecto crezca para meterle Git, ya perdiste el historial de las primeras decisiones, que casi siempre son las que menos te acuerdas después. A mí me quedó claro armando este mismo proyecto en tres computadoras distintas: cada tropiezo con JAVA_HOME quedó, más o menos, registrado en los commits, y el repositorio remoto fue lo que me salvó de tener que dejar las tres máquinas configuradas exactamente igual.

3.2 En la aplicación elegida, ¿qué problema ocurriría si la capa de presentación accediera directamente a la base de datos, sin pasar por la capa de negocio?

Se caerían todas las reglas que hacen que Discord no sea un caos. Cualquiera podría escribir en un canal donde no tiene permiso, porque nadie estaría revisando su rol antes de guardar el mensaje. Tampoco se aplicaría el límite de caracteres ni el filtro anti-spam, así que con un cliente modificado se podría inundar un canal sin que nada lo frene. La capa de negocio es, básicamente, la que hace cumplir las reglas; sin ella la base de datos queda abierta a cualquier cosa, válida o no.

3.3 ¿Qué ventajas ofrece un repositorio remoto como GitHub frente a guardar el código solo en la computadora local?

La más obvia: no depende de una sola máquina. Si el equipo falla o el archivo se pierde, el código sigue ahí. Lo viví tal cual en este laboratorio, saltando entre tres computadoras distintas para el mismo proyecto — sin GitHub cada salto hubiera sido empezar de cero o andar cargando el proyecto en un USB. Además cada commit queda con fecha y autor, lo cual sirve de evidencia para el curso y también de bitácora propia, y es la base para poder trabajar en equipo más adelante sin pisarnos el código.

3.4 Investigue: ¿qué diferencia existe entre un IDE y un editor de texto como Notepad++ o Sublime Text?

Un editor de texto resalta sintaxis y poco más; para compilar o correr el código tienes que salir de ahí y usar otra herramienta por tu cuenta. Un IDE junta en la misma ventana el editor, el compilador, el depurador y la gestión del proyecto. Es literalmente lo que hice para este laboratorio en VS Code con la extensión de Java y Maven: crear el proyecto, escribir las cuatro clases, compilarlas y correrlas, todo sin cambiar de programa ni una vez.

4. Conclusiones

Este laboratorio no era para dominar ningún patrón todavía, pero fue mi primer choque real con la idea de que el entorno de trabajo también cuenta como parte del diseño de software: un JDK equivocado, un .gitignore que falta, un remoto mal apuntado, y ya se te fue tiempo real antes de escribir una sola línea de lógica de negocio. Reconocer las tres capas en Discord, aunque fue solo de forma visual, ya me deja una idea concreta de por qué importa el patrón de capas: cada nivel protege al siguiente de accesos que no debería tener, y eso lo voy a volver a ver, ahora en código, en el Eje Temático N.° 2.